Assignment 3: Reinforcement Learning in the Multi-Elevator Domain

Bar-Ilan CS 89-570
Introduction to Artificial Intelligence

June 13, 2026

1 Introduction

In this exercise you will keep working with the *Multi-Elevator* domain from Assignments 1 and 2. The physical world is exactly the same: several elevators move people inside a building, respecting their reachable-floor sets and weight capacities.[1]

The crucial change in this assignment is that we move to the *Reinforcement Learning* (RL) setting. The world is still the stochastic MDP of Assignment 2 – elevators and people fail with some probability, and successful deliveries give random rewards – but now **the model is hidden from the agent**: the transition probabilities and the reward distributions are *unknown*. The agent can no longer query the success probability of an elevator or the reward list of a person. The only way to find out how the world behaves is to **act and observe the consequences**.

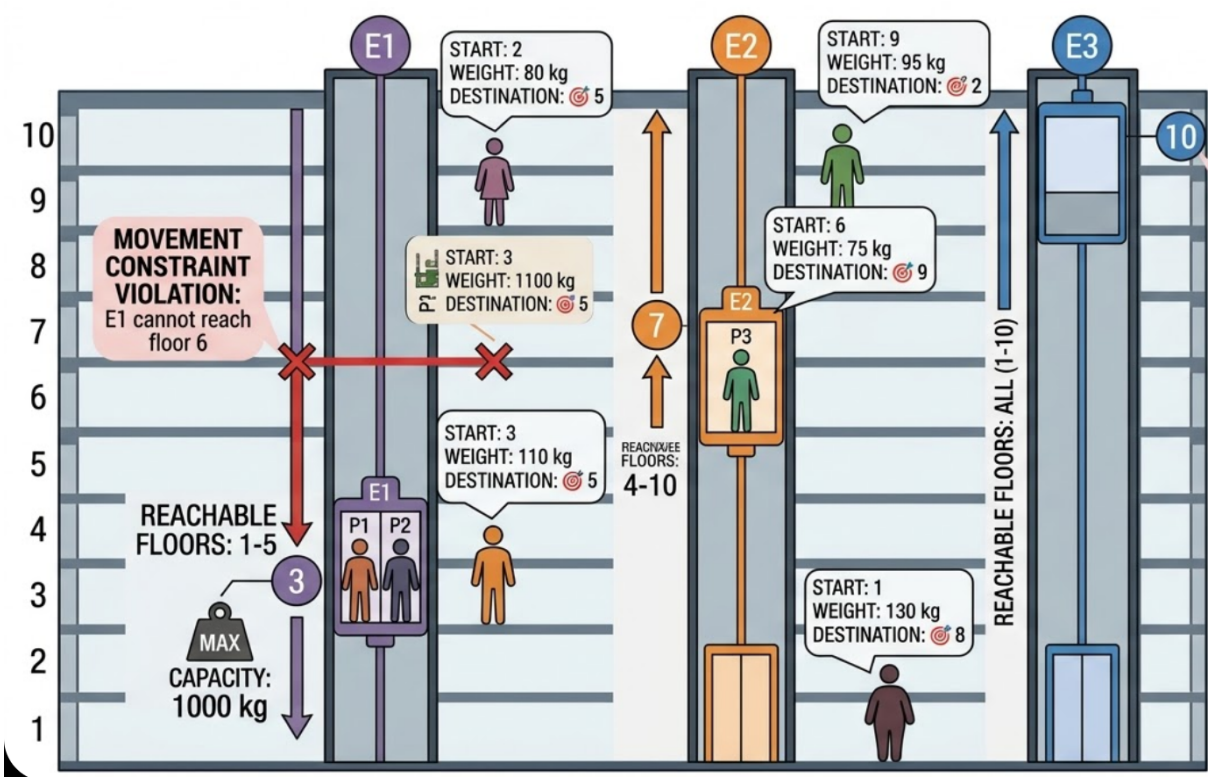


Figure 1: The original *Multi-Elevator* domain from Assignment 1.

[1] See Assignment 1 for the full deterministic version of the domain, and Assignment 2 for the stochastic (MDP) version.

Your task stays the same as in Assignment 2: implement a controller function that, given the current state of the task, chooses the next action, with the goal of **maximizing the expected accumulated reward** over the episode within the provided step budget (*horizon*).

The difference is purely epistemic, but it changes everything about how you should act. In Assignment 2 you knew the model and could plan against it. Here you must *learn* the relevant parts of the model on the fly – which elevators are reliable, which people are easy to move, and which deliveries are worth chasing – while at the same time *using* what you have already learned to collect reward. Balancing these two pressures is the heart of this assignment (see Section 3).

2 Problem Description

2.1 Reminder: deterministic core (Assignment 1)

We work in the same building as before:

- Floors are numbered $0, 1, \ldots,$ `height`. Floor 0 is the ground floor.
- Each elevator has a unique ID, a current floor, a set of floors it can reach, and a maximum weight capacity.
- Each person has a unique ID, a starting floor, a weight, and a destination floor. A person may either stand on some floor or be inside exactly one elevator.

Elevators and people interact through the three basic actions:

- MOVE(ELEVATOR_ID, TARGET_FLOOR): move an elevator from its current floor to another floor in its reachable set.
- ENTER(PERSON_ID, ELEVATOR_ID): move a person from their current floor into an elevator on the same floor. Legal only if the person is not already inside an elevator and the total weight inside the elevator does not exceed its maximum capacity after entering.
- EXIT(PERSON_ID, ELEVATOR_ID): move a person out of an elevator to the elevator's current floor. Legal only if the person is currently inside that elevator.

2.2 Stochastic dynamics with a hidden model

The world is the stochastic MDP of Assignment 2. We restate the dynamics here, but emphasize that the numbers driving them are no longer available to your controller:

- **Defective elevators.** Each elevator e has a success probability for its MOVE action. With this probability the MOVE succeeds; with the remaining probability it fails and the elevator ends up on a different floor (described below). This probability is **unknown** to the agent and is *not* part of the state. It may vary between problems, but it is fixed across episodes of the same problem.
- **Unreliable people.** Each person p has a success probability for ENTER/EXIT actions targeting them. With this probability the action succeeds; with the remaining probability it has no effect. This probability is also **unknown** to the agent and is not part of the state.
- **Random delivery rewards.** Whenever person p *successfully* exits an elevator *on their destination floor*, a reward is drawn from a person-specific distribution and added to the total reward; p is then delivered and removed from the state. The reward distribution (and the list of values it is drawn from) is **unknown** to the agent; all delivery rewards are positive. The only way to learn how much a delivery is worth is to deliver the person and observe the reward you receive.

- **Goal reward.** When *every* person has been delivered, the episode is completed and an additional deterministic `goal_reward` is given. The state is then reset to the initial state and the episode continues; the horizon keeps running. The `goal_reward` *is* known to the agent.
- **Horizon.** The maximum number of steps the controller has to accumulate reward. In this assignment horizons are deliberately large (typically between 250 and 500 steps) so that you have enough budget to both *explore* the unknown model and *exploit* what you learn – see Section 3.

As in Assignment 2, you do not provide a sequence of actions in advance. At each step the controller receives the current state and must decide which action to execute next.

2.3 Actions and their outcomes

For each chosen action, the engine performs the following steps:

1. **Legality check.** First, the task checks that the chosen action is legal in the current state:
   - MOVE(E, F) is legal only if f belongs to the reachable-floor set of e.
   - ENTER(P, E) is legal only if person p and elevator e are on the same floor, p is not already inside any elevator, and the total weight of people inside e after entering does not exceed the elevator's maximum capacity.
   - EXIT(P, E) is legal only if person p is currently inside elevator e.

   If the action is illegal, the engine raises an error – something that should never occur with a correct controller.
2. **Success vs. failure.** If the action is legal, the engine flips a biased coin (whose bias is hidden from you) for that elevator or that person:
   - With the success probability the action *succeeds* and its intended effect is applied.
   - With the remaining probability the action *fails*, leading to a different outcome.

We now specify the exact effect of each action in both cases. The dynamics are identical to Assignment 2; only your knowledge of the probabilities has changed.

move(e, f). Let F_e denote the set of floors reachable by elevator e, and let f_0 be the current floor of e.

- **On success**: elevator e moves from f_0 to f.
- **On failure**: the new floor of e is chosen uniformly at random from

$$\{f_0\} \cup (F_e \setminus \{f\}),$$

  i.e. *stay in place* or *go to any reachable floor other than the intended target.*

enter(p, e).

- **On success**: person p enters elevator e; the carried-weight inside e is increased by the weight of p.
- **On failure**: nothing changes – person p stays on the floor, elevator e is unaffected, and no reward is obtained.

exit(p, e).

- **On success**: person p exits e onto e's current floor; the carried-weight inside e is decreased by the weight of p. Then:
  - if e's current floor equals the destination floor of p, a reward is drawn from p's (hidden) reward distribution and added to the total reward; person p is removed from the state (delivered);
  - otherwise, no reward is obtained and p now stands on e's current floor.
- **On failure**: nothing changes – person p stays inside elevator e and no reward is obtained.

reset. At any step, the controller may choose to reset the state to the initial state. This action always succeeds (probability 1), gives no reward, and costs 1 step. If your step budget was x before applying RESET, it will be $x - 1$ after. The accumulated reward is not reset.

Episode termination and goal reward. Whenever every person has been delivered, the controller receives the additional `goal_reward`, the episode is considered finished, and the task is reset to the initial state. The horizon keeps running across episodes. **Note that every action costs** 1 **step (reset included).**

3 Exploration vs. Exploitation

This is the conceptual core of the assignment, and the reason horizons are large.

Because the model is hidden, your controller faces the classic reinforcement-learning dilemma. At every step it must implicitly choose between two kinds of action:

- **Exploration**: trying actions in order to *learn* the action distribution and the rewards – how reliable each elevator's MOVE is, how likely each person is to ENTER/EXIT, and how much reward each delivery actually yields on average. Exploration may waste steps in the short term, but it improves the quality of your future decisions.
- **Exploitation**: using the estimates you have built so far to take the action you currently believe is best, so as to actually *collect* reward.

A controller that only exploits will lock onto whatever looked good first and may never discover a far better plan (for instance, that a seemingly attractive person is almost impossible to move, or that an alternative elevator is far more reliable). A controller that only explores will keep gathering information it never cashes in, and will score poorly. The art is to explore enough, and then shift toward exploitation as your estimates become trustworthy.

What you can learn, and from which signals. Your only window into the hidden model is the stream of *observations* you get by acting: the state before and after each action, and the reward gained from the last action (provided through the API, see Section 6). From this stream you can, for example:

- estimate each elevator's MOVE success rate by counting how often a MOVE actually lands on the intended floor;
- estimate each person's ENTER/EXIT success rate by counting how often the action takes effect;
- estimate the *average* delivery reward of each person from the rewards you actually observe on successful deliveries.

Some starting ideas (not mandatory). You are free to design any controller as you see fit. Some basic approaches to the exploration–exploitation trade-off that you may find useful include ε-greedy action selection, optimism in the face of uncertainty (e.g., upper-confidence-bound style bonuses), and decaying exploration over the horizon. You may also combine learning with the planning techniques from Assignment 2 once you have estimates of the unknowns.

4 Problem Specification

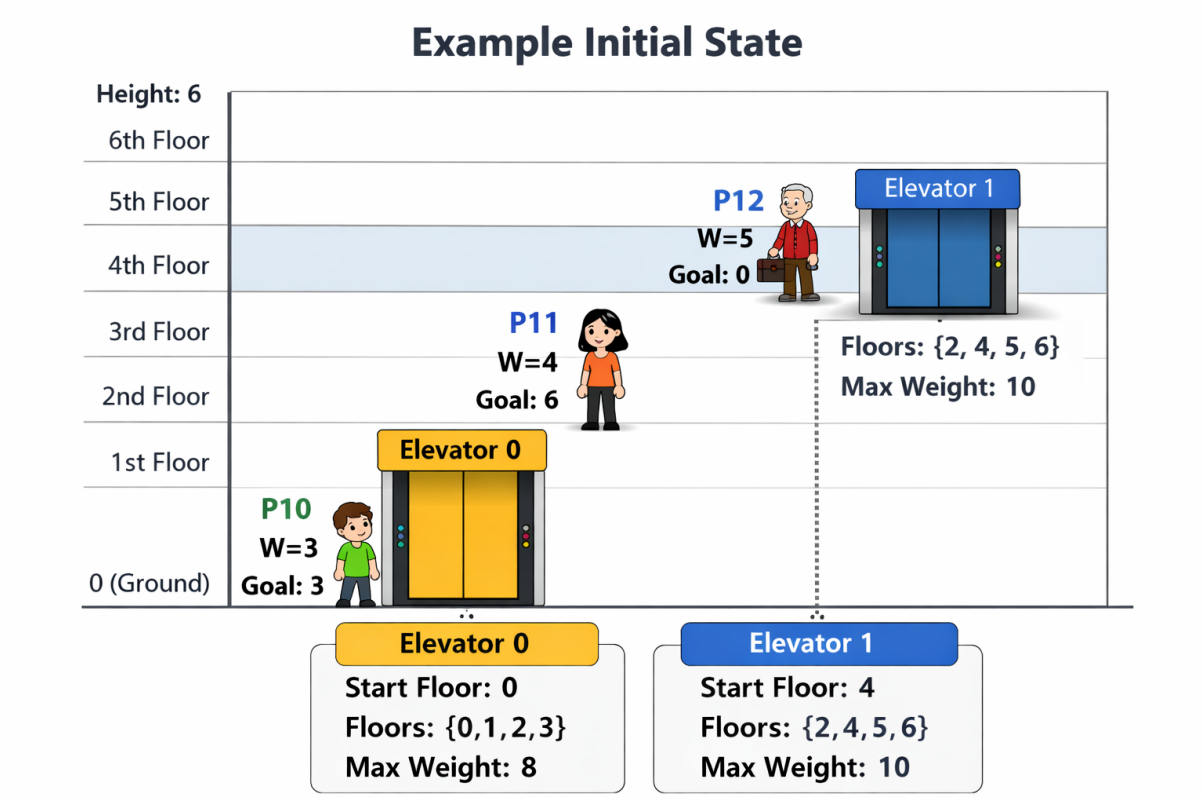
4.1 Problem dictionary

The problem is specified as a Python dictionary `problem`, similar to Assignment 2. The probability dictionaries and the reward lists are still used internally by the engine to run the simulation, but they are **not** exposed to your controller. The entries are:

1. **height** – an integer. The building floors are $\{0, 1, \ldots, \texttt{height}\}$.
2. **Elevators** – a dictionary whose keys are elevator IDs. The value of elevator ID e is a tuple
$$(f, F, w_{\max}),$$
where f is the current floor of the elevator, F is the set of floors this elevator can reach, and $w_{\max}$ is its maximum weight capacity.
3. **Persons** – a dictionary whose keys are person IDs. The value of person ID p is a tuple
$$(f_{\text{start}}, w, f_{\text{goal}}),$$
where f_{start} is the person's starting floor, w is the person's weight, and f_{goal} is the destination floor.
4. **elevator_chosen_action_prob** – a dictionary mapping each elevator ID to its (hidden) MOVE success probability. **Used by the engine only; not accessible from your controller.**
5. **person_chosen_action_prob** – a dictionary mapping each person ID to its (hidden) ENTER/EXIT success probability. **Used by the engine only; not accessible from your controller.**
6. **persons_reward** – a dictionary mapping each person ID to the (hidden) list of possible delivery rewards, from which one value is sampled uniformly at random on a successful delivery. **Used by the engine only; not accessible from your controller.**
7. **goal_reward** – a number giving the (known) reward received when *every* person has been delivered.
8. **seed** – an integer seed used to initialize the engine's random number generator, so that runs are reproducible.
9. **horizon** – the number of steps the controller has to accumulate reward (typically 250–500).

What is hidden. The success probabilities (`elevator_chosen_action_prob`, `person_chosen_action_prob`) and the reward lists (`persons_reward`) are **not** part of the state and **cannot** be queried through the API. Your controller knows only: the geometry (heights, reachable sets, capacities), the static person data (weight, destination), the goal reward, and whatever it observes by acting.

4.1.1 Example




```
problem = {
    "height": 6,
    "Elevators": {
        0: (0, (0, 1, 2, 3), 8),
        1: (4, (2, 4, 5, 6), 10)
    },
    "Persons": {
        10: (0, 3, 3),
        11: (2, 4, 6),
        12: (4, 5, 0)
    },
    # ---- hidden from the agent (engine use only) ----
    "elevator_chosen_action_prob": {0: 0.8, 1: 0.7},
    "person_chosen_action_prob":   {10: 0.9, 11: 0.6, 12: 0.85},
    "persons_reward": {
        10: [2, 3, 6, 10],
        11: [1, 5, 6, 10],
        12: [3, 4, 8, 12]
    },
    # ---- visible to the agent ----
    "goal_reward": 30,
    "horizon": 300
}
```

4.2 Goal description

As in Assignment 2, the goal is to choose actions that **maximize the expected total reward** obtained during the episode, where the reward comes from:

- stochastic rewards when persons are successfully delivered to their destination floors, and
- a deterministic `goal_reward` once every person has been delivered. After receiving the `goal_reward`, the state is reset to the initial state, and the remaining step budget continues to be spent.

Your implementation will not construct a full plan in advance, but will decide, at each step, which action to take based on the current state and on what it has learned so far.

5 State Representation

The state representation is fixed and you must not change it (it is identical to Assignment 2):

`(elevators_t, persons_t, total_persons_remaining)`

Where:

- `elevators_t` is a tuple of elevators. Each elevator is represented as $(eid, f, w_{\text{load}})$, where eid is the elevator's ID, f is the elevator's current floor, and w_{load} is the total weight currently inside the elevator.
  Note: you can obtain an elevator's maximum capacity and reachable set using provided functions.
- `persons_t` is a tuple of persons that have *not yet been delivered*. Each person is represented as (pid, loc), where pid is the person's ID and loc encodes the person's current location:
  - $loc = (\texttt{'floor'}, f)$ – the person is standing on floor f;
  - $loc = (\texttt{'in'}, eid)$ – the person is inside elevator eid.

  Note: a person's starting floor, weight, and destination are static and can be obtained using provided functions. The per-person reward list is hidden and cannot be obtained.
- `total_persons_remaining` is the number of persons that have not yet been delivered.

6 Engine API

Your controller interacts with the engine through a single object of type `GameAPI`. The full set of methods you may call is listed below; any access beyond this list is forbidden (see the engine-access policy at the end of this section). Each method returns an *immutable copy* of internal data when relevant, so mutating a returned object has no effect on the engine.

Compared with Assignment 2, the three getters that revealed the hidden model have been **removed**: `get_elevator_action_prob`, `get_person_action_prob`, and `get_person_reward`. One getter has been **added** to support learning: `get_last_gained_reward`.

- Returns the current state tuple (see Section 5):

```
def get_current_state(self):
    return self._state
```

- Returns the initial state tuple – the state the task is reset to after a successful goal episode or after RESET:

```
def get_initial_state(self):
    return self._initial_state
```

- Returns whether the task is finished (e.g., when `steps == max_steps`):

```
def get_done(self):
    return self._done
```

- Returns the number of steps performed so far:

```
def get_current_steps(self):
    return self._steps
```

- Returns the horizon of the problem (maximum number of steps):

```
def get_max_steps(self):
    return self._max_steps
```

- Returns the current accumulated reward obtained from the actions taken so far:

```
def get_current_reward(self):
    return self._reward
```

- Returns the reward gained from the *last* action performed (0 if the last action yielded no reward). This is your main learning signal – use it to estimate the unknown reward distributions over time. **Note:** on the delivery that completes the global goal, this value also includes `goal_reward` (which you know via `get_goal_reward`), so subtract it to recover the pure delivery sample:

```
def get_last_gained_reward(self):
    return self._last_gained_reward
```

- Returns the deterministic reward awarded when every person has been delivered:

```
def get_goal_reward(self):
    return self._goal_reward
```

- Returns the weight of person p:

```
def get_person_weight(self, p):
    return self._person_weight[p]
```

- Returns the destination floor of person p:

```
def get_person_goal(self, p):
    return self._person_goal[p]
```

- Returns a fresh dictionary of elevator capacities (`eid : max_weight`):

```
def get_capacities(self):
    return dict(self._capacities)
```

- Returns a fresh dictionary of elevator reachable-floor sets (`eid : frozenset`):

```
def get_reachable(self):
    return dict(self._reachable)
```

Engine access policy. Your controller may interact with the engine **only** through the getter methods listed above and `submit_next_action`. The following are strictly forbidden and will result in a grade of **zero** for this assignment:

- Accessing any attribute of the engine object whose name starts with an underscore (single `_` or double `__`). In particular, do not try to read the hidden probabilities or reward lists.
- Modifying any data returned by a getter and assuming the engine reflects the change.
- Re-seeding, replacing, or otherwise manipulating the engine's random number generator (including `np.random.seed`, `np.random.default_rng`, or shadowing the `numpy` module).
- Modifying probabilities, rewards, capacities, reachable sets, or any other problem parameter at runtime.
- Importing `ext_elev` internals beyond the documented API, monkey-patching its classes or functions, or using `vars()`, `__dict__`, `gc`, `ctypes`, or similar introspection facilities to inspect or modify engine state.

Submissions will be inspected for these patterns. No warning is given – the first occurrence is sufficient for a zero.

7 What You Must Implement

We provide a starter codebase. Most of the infrastructure is already implemented. Your job is to complete the methods so that the checker can run instances.

Files and required functions (in `ex3.py`)

You *must* implement the following (you may add helpers as you see fit):

1. **`choose_next_action(state) -> str`**
    - **Input:** a state tuple `state` = (`elevators_t`, `persons_t`, `total_persons_remaining`).
    - **Output:** a *single* legal action encoded as a string, in the same format used in Assignments 1 and 2:
        - MOVE(E, F): `MOVE{e,f}`
        - ENTER(P, E): `ENTER{p,e}`
        - EXIT(P, E): `EXIT{p,e}`
        - RESET: `RESET` (with no arguments)
    - **Possible actions (depending on legality):** MOVE, ENTER, EXIT, RESET.

Because the model is hidden, a sensible controller will maintain its own internal estimates (success counts, observed rewards, etc.) across calls to `choose_next_action`, update them using the latest observation, and use them to decide between exploration and exploitation.

8 Scoring & Evaluation

- There will be **X problem instances** where $X \geq 4$.
- Each instance will be evaluated using **30 different random seeds**. For each seed, we run the instance once, record the total reward, and then compute the instance score as the average reward across all tested seeds (sum of rewards divided by the number of seeds).
- **Per-seed time budget:** each seed has a wall-clock budget of $20 + 0.5 \cdot$ `horizon` seconds for your controller's actions. **If a seed exceeds its time budget, that seed receives a reward of** 0 **for grading purposes**, regardless of any reward accumulated up to that point. The instance score is the average reward across the 30 seeds, with timed-out seeds counted as zero.
- To receive a passing grade (60):
  - For each problem in the test set provided with the starter code, **your average reward must be higher than that of a random policy**.
  - There is a **validation check in `ext_elev.py`**. Make sure that every action you return from `choose_next_action` passes this check.
- To receive a grade higher than 80, your code must surpass the performance of our baseline, which we will publish next week.
- The last 20% of the grade will be determined on the basis of performance. *This is not a competition: more than one student can receive 100.*

9 Attached files

The starter code includes the following files:

1. **`ex3.py`** – *the only file you submit, and the only file that will be graded.* Implement `__init__`, `choose_next_action`, and set your `id` variable. Do not rename or move this file (except to `ex3_id.py` at submission time).
2. **`ex3_check.py`** – a local *self-checker*. Use it to run your code on the provided instances. You may add additional problems here for your own testing, but the grading system will *not* use your added problems. Changes to this file do not affect grading.
3. **`ext_elev.py`** – the engine that runs your action-selection function on the current state. You interact with it only through the `GameAPI` object documented in Section 6. You do not submit this file; do not modify it.
4. **`ex3_random.py`** – a *random baseline controller*. Picks uniformly at random from the legal actions at each step. This is the policy you must beat to receive a passing grade.
5. **`ex3_gui.py`** – an optional *visualizer* built with `pygame`. Renders the building, elevators, and persons step-by-step so you can watch your controller play. Not used for grading.

Running the code

All commands are run from the directory containing the starter files.

Self-checker. Runs your controller across all provided instances and prints average rewards:

```
python ex3_check.py
```

Random baseline. To measure the random policy on the same instances (useful for sanity-checking the bar you need to clear), edit `ex3_check.py` and replace `import ex3` with `import ex3_random as ex3`, then run it as above.

Visualizer. Requires `pygame` (`pip install pygame`). Launch with:

```
python ex3_gui.py
```

A window opens showing the building. Controls:

- `SPACE` – advance one step
- `1` – run one full episode
- `0` – run all episodes back-to-back
- `ESC` – quit

By default the GUI drives the engine with `ex3_random`. To watch your own controller, change the top-of-file import `import ex3_random as student` to `import ex3 as student`. You can also swap the `problem` dict at the top of the file to visualize a different instance.

10 Rules & Submission

- **Edit only `ex3.py`**, and rename it to `ex3_id.py` (e.g. `ex3_123456789.py`) before submission. Do not modify or submit any other file.
- **Individual work.** Discussing ideas is allowed, but *sharing or copying code is prohibited.*
- **Use of AI / online sources.** You may use them, but you *must* clearly note *everywhere* you used them and for what purpose. Failure to disclose will reduce your grade.
- **Academic integrity.** We take plagiarism seriously, please do not make us deal with it.
- **Questions.** Ask during office hours or in the assignment forum only.
- **Extensions.** No extensions will be granted except for documented exceptional circumstances via email.
- **Deadline:** 30/6/2026.
- **Identification.** Set your ID in the variable `id` in `ex3.py`.
- **What to submit & where:** Submit `ex3.py`, but rename it before submission to `ex3_id.py`, where `id` is your student ID. Upload the renamed file using the Bar-Ilan `submit` system: `https://submit.cs.biu.ac.il/cgi-bin/welcome.cgi`.

Hard rule Each seed is executed independently. For every seed, your files are rerun from scratch, including the initialization step.

Your controller must not share, save, load, or reuse any information across different seeds. Any attempt to pass data between seeds is not allowed.

For each seed, the controller must learn and interact with the environment as if it is seeing it for the first time.

Rules (summary).

- Unit action cost (each action, RESET included, costs one step).
- At each step, exactly one action is executed.
- ENTER requires the person and the elevator to be on the same floor, and weight capacity must be respected.
- EXIT requires the person to be currently inside that elevator.
- MOVE requires the target floor to be in the elevator's reachable set.
- Each person starts standing on their starting floor with no elevator assignment.
- The transition probabilities and reward distributions are hidden; learn them by acting and observing `get_last_gained_reward`.
- The goal is to maximize the reward accumulated within the given horizon (250–500 steps).